Lab 16 — Answers with commentary

Every answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can check at the terminal.

Question 1 — Requests and limits

Answer. The **request** is read by the **scheduler**: it picks a node whose remaining capacity (capacity minus the sum of requests already placed) covers the demand. The **limit** is written by the **kubelet** into the container's **cgroup**. Memory beyond the limit: the kernel kills the process (`OOMKilled`, code 137), the kubelet restarts it (`RESTARTS` + 1, then `CrashLoopBackOff`). CPU beyond the limit: the cgroup **slows down** the container (*throttling*), nothing is killed.

Why. Memory is not "lent": an allocated page stays allocated until freed; CPU is shared in time slices.

Nuance. The CPU request also drives proportional sharing (`cpu.weight`) when the node is saturated; the memory request feeds the eviction computation. Without a memory limit, one container can drain the node and trigger the eviction of other Pods.

Example.

```
kubectl describe node | sed -n '/Allocated resources/,/Events/p'   # Requests  Limits  per resource
kubectl get pod glouton -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'   # OOMKilled
```

Question 2 — QoS and eviction order

Answer. `db` is `Guaranteed` (requests = limits), `api` is `Burstable`, `batch` is `BestEffort`. Under memory pressure, the kubelet evicts the `BestEffort` first (`batch`), then the `Burstable` ones exceeding their request (`api`), and the `Guaranteed` last. If `batch` were a critical job, it would be the first sacrificed, simply because nobody declared what it consumes.

Why. The kubelet has no notion of business importance; it only knows what the spec declares. A Pod without `resources` is, by definition, a Pod one can do without.

Nuance. QoS only governs eviction by the **kubelet**; the kernel's OOM killer acts per cgroup when a limit is exceeded, whatever the class. For an important batch: requests = limits, or at least realistic requests, plus a `PriorityClass` if the platform offers one.

Example.

```
kubectl get pods -o custom-columns=NAME:.metadata.name,QOS:.status.qosClass
```

Question 3 — `OOMKilled`

Answer. `kubectl describe pod glouton` shows `Reason: OOMKilled`, `Exit Code: 137` and the limit (`Limits: memory: 32Mi`); `kubectl get pod glouton -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'` gives `OOMKilled`. `RESTARTS` grows because `restartPolicy: Always` (the default) makes the kubelet relaunch the container after every death, with a growing delay (`CrashLoopBackOff`). Here the application is

designed to allocate 100 MB under a 32 Mi limit: both need revisiting — but in real life it is almost always **the application** (leak, badly sized JVM heap) or a **limit that is too tight**, not a limit "too low in itself".

Why. 137 = 128 + 9: the process received `SIGKILL` from the kernel (lab 03, lab 10).

Nuance. An `OOMKilled` can also hit a container **without** a limit: if the whole node runs out of memory, the kernel kills the biggest process. `describe node` then shows `MemoryPressure True`.

Example.

```
kubectl describe pod glouton | grep -E 'Reason:|Exit Code|Restart Count'
#       Reason:       OOMKilled  /  Exit Code:    137  /  Restart Count:  1
```

Question 4 — Three Pods `Pending`

Answer. The scheduler no longer finds a node whose **reservable** memory covers one more request; the Pods stay `Pending` with `FailedScheduling: 0/1 nodes are available: 1 Insufficient memory`. Possible fixes: (1) **lower the request** of the API if it is overestimated (measure with `kubectl top pods`); (2) **add a node** or give the node more memory (`.wslconfig` on WSL); (3) **free** the node of other Pods (or reduce replicas). The most honest is (1) **if** the measurement shows the API runs well below its request; otherwise it is (2): a realistic request is not a setting, it is a fact.

Why. Placement is based on **requests**, not on real consumption: a node can be "full" at 95% of requests with 20% utilisation.

Nuance. Lowering the request below real usage turns the Pod into an over-consuming `Burstable`: it will be evicted first under pressure. A managed cluster adds a node automatically (*cluster autoscaler*); k3s on WSL does not.

Example.

```
kubectl describe pod trop-gourmand | grep FailedScheduling
#  Warning  FailedScheduling  … 0/1 nodes are available: 1 Insufficient memory. …
```

Question 5 — LimitRange

Answer. (a) The Pod receives `requests: {memory: 32Mi}` and `limits: {memory: 128Mi}` injected into its spec at creation (visible in `kubectl get pod -o yaml`). (b) The Pod is **rejected** at `apply`: `maximum memory usage per Container is 1Gi, but limit is 4Gi`. (c) The Pod keeps its `64Mi` request and receives `limits.memory: 128Mi` by default. The platform team imposes these objects so that no Pod runs without declared resources (hence in `BestEffort`) and none can reserve an entire node.

Why. The LimitRange acts as an **admission controller**: it modifies (defaults) or rejects (max/min) every Pod at creation time.

Nuance. The LimitRange does not touch Pods already created; and if `defaultRequest` exceeds `default`, the object is invalid. A `default` without `defaultRequest` yields request = limit (hence `Guaranteed`).

Example.

```
kubectl run sans-limites --image=docker.io/library/alpine --command -- sleep 3600
kubectl get pod sans-limites -o jsonpath='{.spec.containers[0].resources}'
# {"limits":{"cpu":"250m","memory":"128Mi"},"requests":{"cpu":"50m","memory":"32Mi"}}
```

Question 6 — ResourceQuota exceeded

Answer. The remaining `requests.memory` budget is `1Gi − 3×64Mi = 832Mi`; eight Pods of 128Mi ask for 1024Mi. The **ReplicaSet** creates Pods one by one: six go through (768Mi), the seventh is refused. `kubectl get pods` shows **six** Pods of the second Deployment, not eight; `kubectl describe deployment` says `6/8` (`ReplicaFailure True`), and the refusal is read in **`kubectl describe replicaset`** (or `kubectl get events`): `exceeded quota: budget, requested: requests.memory=128Mi, used: 896Mi, limited: 1Gi`.

Why. The quota is checked at the creation of every **Pod** by the API server; the Deployment itself is accepted — it consumes nothing.

Nuance. The `pods: 10` bound would also play: 3 + 8 = 11 > 10, so at most seven Pods anyway. The ReplicaSet retries indefinitely: as soon as budget is freed, the missing Pods appear.

Example.

```
kubectl describe quota budget | sed -n '/Resource/,$p'
# requests.memory  160Mi  1Gi  …
```

Question 7 — `logs --previous`

Answer. `kubectl logs` shows the **current** container: after a restart, that is a new process whose logs start at boot. The cause of death is in the output of the **previous** container, which the kubelet keeps for a single generation: `--previous`. `lastState` shows `{"terminated":{"reason":"OOMKilled"|"Error"|"Completed","exitCode":137,"finishedAt":…}}`: the reason and code of the last dead container.

Why. Every restart creates a new container; the runtime keeps the previous one (and its log) until the next.

Nuance. Two restarts later, the first log is gone: hence centralised log collection (Loki, Elastic) in production. And `--previous` fails on a Pod that has never restarted.

Example.

```
kubectl exec $POD -- sh -c 'kill 1' ; sleep 5
kubectl logs $POD --previous | tail -1     # API arretee proprement.
```

Question 8 — `kubectl debug --target`

Answer.

```
kubectl debug $POD -it --image=docker.io/library/alpine --target=api --profile=general -- sh
```

`--target=api` places the ephemeral container in the **process namespace** of the `api` container: `ps` shows the `java -jar`, `/proc/1/root/` gives access to its file system, and `localhost:8080` is its port (the Pod's network namespace). It beats a shell in the image because the image stays

minimal and tool-less for an attacker (lab 05), and because the diagnostic shell is chosen **when** it is needed, with the right tools (`netshoot` for the network, `alpine` for a `ps`).

Why. Ephemeral containers are an `ephemeralContainers` field of the Pod spec, added without restarting the Pod; they cannot be removed, but vanish with it.

Nuance. Without `--target`, the ephemeral container shares the network but not the processes. `kubectl debug node/wsl-ubuntu` likewise opens a shell on the node. A `--profile=legacy is deprecated` message appears if you omit `--profile`.

Example.

```
kubectl debug $POD -it --image=docker.io/library/alpine --target=api -- sh -c 'ps aux | head -3; wget -qO- localhost:8080/actuator/health'
#     1 1000      0:01 java -jar /app/api.jar   …   {"status":"UP"}
```

Question 9 — `Forbidden`

Answer. `Error from server (Forbidden)`: the API server authenticated the caller and then refused authorisation. `User "system:serviceaccount:lab16:reader"`: the subject is the ServiceAccount `reader` of namespace `lab16` (prefix `system:serviceaccount:`). `cannot list resource "secrets" in API group ""`: the verb `list`, the resource `secrets`, in the *core* group (empty). `in the namespace "lab16"`: the scope tested. It is **the intended behaviour**: the Role lists only `pods` and `pods/log`. For ConfigMaps: add a rule `resources: ["configmaps"], verbs: ["get", "list", "watch"]` — and definitely not `secrets`.

Why. RBAC is **additive and closed by default**: anything not explicitly granted is refused.

Nuance. `--as` requires the `impersonate` right, which `cluster-admin` has; that is what lets you test without creating a kubeconfig. The built-in `view` ClusterRole also excludes Secrets, on purpose.

Example.

```
kubectl auth can-i get secrets --as=system:serviceaccount:lab16:reader    # no
kubectl auth can-i get pods    --as=system:serviceaccount:lab16:reader    # yes
```

Question 10 — Role vs ClusterRole

Answer. Role + RoleBinding: rights on the resources **of one namespace** (the billing team may deploy in `billing-prod`). **ClusterRole + ClusterRoleBinding**: rights **over the whole cluster**, including non-namespaced resources (nodes, PVs, namespaces): the platform team, a monitoring agent reading all Pods. **ClusterRole + RoleBinding**: a role defined **once** at cluster level but **granted in a single namespace**: that is how you give `edit` on `billing-prod` without redefining it everywhere. The `view` / `edit` / `admin` ClusterRoles exist precisely to be bound by RoleBinding, namespace by namespace.

Why. A RoleBinding always restricts the referenced role to its own namespace, whatever the role's kind.

Nuance. `admin` through a RoleBinding allows managing the namespace's RBAC, but not creating namespaces or touching quotas. `cluster-admin` through a ClusterRoleBinding is the key to everything: reserved for the platform's humans, never for an application ServiceAccount.

Example.

```
kubectl create rolebinding dev-edit --clusterrole=edit --user=alice -n billing-prod
kubectl auth can-i create deployments --as=alice -n billing-prod   # yes
kubectl auth can-i create deployments --as=alice -n default        # no
```

Question 11 — NetworkPolicy

Answer. First `wget`: failure (`wget: can't connect to remote host (10.43.x.x): Connection refused`, or *timeout*). Second, after the `app=web` label: `{"status":"UP"}`. As soon as a NetworkPolicy **selects** a Pod (`podSelector: app=api`), all inbound traffic not listed is refused; before, with no policy, everything passed. On a cluster whose CNI does not enforce policies, the object is accepted by the API and **ignored**: both `wget` succeed, without any error message.

Why. The NetworkPolicy is declarative; the CNI (kube-router in k3s) translates it into filtering rules on the node, reading labels continuously — hence the immediate effect of a `kubectl label`.

Nuance. The policy only looks at Pods: the ClusterIP is translated before filtering, and the source IP is the client Pod's. Depending on the CNI, the refusal shows as a `refused` or as silence (`timeout`); always test with `--timeout`.

Example.

```
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health   # failure
kubectl label pod client app=web
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health   #
{"status":"UP"}
```

Question 12 — Policies for the stack

Answer. Two `ingress` policies: (1) `podSelector: app=db`, `from: podSelector app=api`, port 5432; (2) `podSelector: app=api`, `from:` two sources, `podSelector app=web` **and** Traefik, port 8080. The difficulty: Traefik runs in `kube-system`, and a `podSelector` alone only looks at the current namespace; you need a `namespaceSelector` (label `kubernetes.io/metadata.name: kube-system`) **combined** with Traefik's `podSelector` in the **same** `from` element (two separate elements would be an OR). One starts with a *deny-all* policy (`podSelector: {}` with no `ingress`) because it makes refusal the **default**: a new Pod is reachable only if someone decided so.

Why. Without a policy selecting it, `web` stays reachable by everyone — intended, since it is served by the Ingress; but any other Pod added tomorrow would be open.

Nuance. A *deny-all* on `egress` also cuts **DNS**: you must allow `kube-system`/CoreDNS on 53 UDP/TCP, a classic trap. And the kubelet's probes are not concerned: they come from the node, not from a Pod.

Example.

```
ingress:
  - from:
      - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system } }
        podSelector: { matchLabels: { app.kubernetes.io/name: traefik } }
      - podSelector: { matchLabels: { app: web } }
    ports: [ { port: 8080 } ]
```

Question 13 — Stolen token

Answer. With `default` and no RoleBinding: the token is valid (it authenticates) but **authorises nothing**; the attacker gets at best `kubectl auth can-i --list` and a few anonymous reads (`/version`, discovery). With `default` bound to `cluster-admin`: they get **the whole cluster**: read every Secret in every namespace, create a privileged Pod on any node, hence the node itself. The two measures from the lab: a **dedicated ServiceAccount with no rights** (and `automountServiceAccountToken: false`, since the API does not call the API server), and an `egress` **NetworkPolicy** preventing the Pod from reaching anything but what it must (the API server included, if nothing justifies it).

Why. The token is a file readable by the process (`/var/run/secrets/kubernetes.io/serviceaccount/token`); whatever the process can do, the attacker running it can do too.

Nuance. Mounted tokens are nowadays **projected** and time-limited (one hour, renewed by the kubelet), which limits exfiltration; but a token valid for one hour with `cluster-admin` is more than enough. `kubectl get clusterrolebindings -o wide | grep cluster-admin` is the basic audit.

Example.

```
kubectl get clusterrolebindings -o wide | grep -w cluster-admin        # who is root of the cluster?
kubectl get pod $POD -o jsonpath='{.spec.serviceAccountName}'          # default?
```